

点灯游戏的结构化求解算法

从首行递推到 $GF(2)$ 多项式、扩展欧几里得与 $O(n^2)$ 求解

王正 | 项目资料整理稿 | 2026 年 4 月

摘要

点灯游戏 (Lights Out) 可以看成定义在 $GF(2)$ 上的线性方程组。朴素高斯消元需要处理 n^2 个按钮变量，矩阵规模为 $n^2 \times n^2$ ，时间复杂度达到 $O(n^6)$ 。项目中的算法利用棋盘的行列递推结构，把二维问题压缩到首行按钮向量，再进一步把首行矩阵表示为邻接扩散矩阵 H 的多项式，从而把求解问题转化为 $GF(2)[x]$ 上的多项式最大公因子和模逆问题。

本文按照项目文件中的 H 系列实现和数学推导，系统整理该算法：先给出点灯游戏的线性模型和首行叠加法，再介绍十字偶校验约束、Krylov 扩散基矩阵 K 、解耦矩阵 F 、Fibonacci 型多项式 $f_n(x)$ 、平移多项式 $c_n(x)=f_n(x+1)$ ，最后说明如何通过扩展欧几里得得到 $q(x)$ 与 $g(x)$ ，并分别处理 $g=1$ 的可逆情况与 $g \neq 1$ 的奇异情况。

算法最终可在 $O(n^2)$ 时间内求出一个完整 $n \times n$ 按钮解；若完整结果流式输出而不常驻内存，工作空间可压缩到 $O(n)$ 。由于完整输出本身就包含 n^2 个格子， $O(n^2)$ 已经达到完整输出问题的渐近下界。进一步优化的主要方向不是绕过输出下界，而是压缩内部多项式运算、统一 ApplyPoly 原语、利用 $D=g(H)$ 的带宽，以及研究 half-gcd、商环统一解法等高阶改造。

关键词：点灯游戏； $GF(2)$ ；首行叠加法；Krylov 矩阵；Fibonacci 多项式；扩展欧几里得；反向消元； $O(n^2)$

目录

- 引言：为什么点灯游戏不是一个普通暴力题
- $GF(2)$ 线性模型与基础算法复杂度
- 首行叠加法：把 n^2 个未知数压到 n 个未知数
- 十字偶校验约束与矩阵生成优化
- 多项式化： H 、 K 、 F 与 $f/c/g/q$
- $O(n^2)$ 主算法：可逆分支与奇异分支
- $g(x)$ 的结构、解的数量与秩缺陷
- 正确性说明与复杂度分析
- 工程实现演进与后续优化方向
- 结论

1 引言：为什么点灯游戏不是一个普通暴力题

点灯游戏的规则非常简单：在 $n \times n$ 棋盘点击一个格子，会翻转该格及其上下左右相邻格子的灯。若从全暗出发，希望所有灯都亮起来，就需要找出每个按钮是否点击一次。由于同一个按钮按两次等价于没按，且不同按钮的点击顺序不影响最终结果，整个问题天然落在 $GF(2)$ 上。按钮状态、灯状态都是 0/1，叠加就是异或。

直观上，这似乎只是一个搜索问题：枚举所有按钮组合，检查灯是否全亮。但 $n \times n$ 棋盘有 n^2 个按钮，完全枚举需要 $2^{(n^2)}$ 种状态。即使只枚举第一行，也仍有 2^n 个候选。项目中的算法真正有意思的地方在于：它没有停留在“怎么搜得少一点”，而是把棋盘结构打扁成一维递推，再把一维递推转化为多项式算子问题。

本文讨论的是方形 $n \times n$ 点灯游戏的一个结构化求解器。它的目标不是只数解，也不是只判断有无解，而是在完整输出按钮矩阵的前提下达到 $O(n^2)$ 时间。因为最终输出本身就有 n^2 个格子，这个复杂度已经很接近“地板价”。再往下优化，只能优化求第一行、求秩缺陷、或内部多项式运算；如果仍要求完整输出，最终总成本还是回到 $O(n^2)$ 。

2 $GF(2)$ 线性模型与基础算法复杂度

2.1 矩阵建模

把棋盘拉直编号后，按钮状态可写为向量 $x \in F_2^{(n^2)}$ ，灯的目标变化量可写为向量 $b \in F_2^{(n^2)}$ 。存在一个 $n^2 \times n^2$ 的 0/1 矩阵 A ，使得 A 的第 j 列表示“只按第 j 个按钮”会翻转哪些灯。于是点灯问题就是线性方程：

$$Ax = b \text{ over } GF(2)$$

如果目标是从全暗变成全亮，则 b 是全 1 向量。静默操作是满足 $Ax=0$ 的非零按钮图案，它不会改变任何灯。若 A 的秩为 r ，则解空间维数为 n^2-r ；只要存在一个特解，全部解就是该特解加上静默空间。

由于棋盘上的翻转关系自反且对称，点灯矩阵 A 是对称矩阵且对角线为 1。对于任意静默向量 x ，有 $x^T Ax = 0$ ；在 $GF(2)$ 中非对角项成对抵消，对角项留下 $\sum x_i$ ，因此任意静默向量的 1 的个数为偶数。由 $(\text{Ker } A^T)^\perp = \text{Im } A$ 可知，全 1 目标属于 A 的像空间，所以这类点灯问题存在解。

2.2 基础路线对比

方法	核心思路	时间复杂度	空间复杂度	评价
完全穷举	枚举 n^2 个按钮的所有 0/1 组合	$O(2^{(n^2)} \cdot n^2)$	$O(n^2)$	最直观，指数爆炸
首行穷举	只枚举第一行，后续行由上一行灯状态唯一决定	$O(2^n \cdot n^2)$	$O(n^2)$	经典压灯法，仍是指数级
全局高斯消元	对 $n^2 \times n^2$ 的点灯矩阵做 $GF(2)$ 消元	$O(n^6)$	$O(n^4)$	能直接得到逆/零空间，但规模太大
首行叠加法	把末行灯表示成首行按钮的线性组合	$O(n^3)$	$O(n^2)$	把未知量从 n^2 压到 n
多项式 + 反向消元	用 f, c, g, q 与 $D=g(H)$ 结构化求解	$O(n^2)$	$O(n)$ 工作空间	本文主线

3 首行叠加法：把 n^2 个未知数压到 n 个未知数

压灯法的关键观察是：第一行按钮一旦确定，第二行按钮就被第一行灯状态唯一决定；第三行按钮又被第二行灯状态唯一决定；如此递推到最后一行。首行叠加法把这个观察线性化：不再枚举第一行，而是直接把最后一行灯表示为第一行按钮的线性组合。

设 $B(n,x)$ 表示第 n 行第 x 列按钮表达式， $L(n,x)$ 表示灯表达式。灯由上、左、中、右按钮叠加得到，而下一行按钮用于修正当前灯。把每一步的固定翻转抽离后，按钮和灯都满足同一种二阶递推：

$$B(n,x)=B(n-1,x-1)\oplus B(n-1,x)\oplus B(n-1,x+1)\oplus B(n-2,x)$$

$$L(n,x)=L(n-1,x-1)\oplus L(n-1,x)\oplus L(n-1,x+1)\oplus L(n-2,x)$$

$$Y(n,x)=\sim(Y(n-1,x-1)\oplus Y(n-1,x)\oplus Y(n-1,x+1)\oplus Y(n-2,x))$$

其中 Y 是翻转向量：它记录“除了首行按钮的线性叠加之外，还需要额外翻转”的常量项。最后一行灯可写为：

$$L_last = M X \oplus Y_last$$

X 是第一行按钮向量， M 是首行按钮到末行灯的 $n \times n$ 线性映射。目标全亮时只需解 $M X = 1 \oplus Y_last$ 。这样，问题从 n^2 个未知数的方程组，降为 n 个未知数的方程组。用普通 GF(2) 消元即可得到 $O(n^3)$ 算法。

这一步是整个项目算法的“第一刀”。它还没有用到多项式 gcd，却已经把规模从 $n^2 \times n^2$ 砍到 $n \times n$ 。后面的 $O(n^2)$ 算法，本质上是在继续榨干 M 的结构，而不是再对 M 做普通消元。

4 十字偶校验约束与矩阵生成优化

项目中的“优化生成矩阵”章节抓住了一个非常漂亮的结构：相关按钮/灯映射矩阵满足十字偶校验约束。对矩阵内部位置，有：

$$B(x-1,y)\oplus B(x+1,y)\oplus B(x,y-1)\oplus B(x,y+1)=0$$

这意味着只要知道某个矩阵的第一行，就能通过递推生成其余行：

$$B(x,y)=B(x-1,y-1)\oplus B(x-1,y+1)\oplus B(x-2,y)$$

直觉上说，矩阵内部的每个点不是自由的，它被上下左右四点的偶校验关系锁住了。因此不需要为每个灯单独生成完整矩阵，只需求关键第一行，再用十字递推补全。这也是后续 $D=g(H)$ 可滚动生成、可带状回代的结构基础。

十字偶校验约束还有一个工程价值：它把“矩阵存储问题”转化为“少数滚动行问题”。对于完整按钮矩阵 t ，如果最终只是输出或验证最后一行，可以边生成边消费，不必常驻整张矩阵。这个思想贯穿了 H15 以后的一系列空间优化。

5 多项式化：H、K、F 与 $f/c/g/q$

5.1 左右扩散矩阵 H

定义 $n \times n$ 矩阵 H ，使 $H(y,x)=1$ 当且仅当 $|x-y|=1$ 。向量 v 乘 H 等价于左右扩散：

$$(vH)[x] = v[x-1] \oplus v[x+1]$$

H 是把二维棋盘压缩后一维递推的核心算子。任何只由“左右扩散若干次再异或”组成的操作，都可以表示为某个多项式作用在 H 上，即 $a(H)$ 。

5.2 Krylov 扩散基 K 与解耦矩阵 F

令 K 的第 i 行为 $e_0 H^i$ ，也就是从第一列单位向量出发扩散 i 次的结果。K 是 Krylov 扩散基矩阵，满足：

$$K(i,x)=K(i-1,x-1)\oplus K(i-1,x+1)$$

$F=K^{-1}$ 是解耦矩阵。它的每一行对应 Fibonacci 型多项式 $f_i(x)$ 的系数。项目中采用的统一记号为：

$$f_0(x)=1, f_1(x)=x, f_n(x)=x \cdot f_{n-1}(x)\oplus f_{n-2}(x)$$

这一组 $f_n(x)$ 既是递推多项式，也是 H 的特征/模多项式体系。对 n 阶 H 来说，有 $f_n(H)=0$ 。这一点非常关键，因为它允许我们在商环 $GF(2)[x]/(f_n)$ 中做计算，然后把多项式代回 H。

5.3 $c(x)=f(x+1)$ 与 p 的省略

首行矩阵 B 可以表示为某个多项式作用在 H 上。早期推导里会引入 $p(x)$ ，并有 $p(x)=f(x)\oplus c(x)$ 。但由于 $f(H)=0$ ，代入 H 后：

$$p(H)=f(H)\oplus c(H)=c(H)$$

因此实际算法不需要显式计算 p，直接用 c 即可。更重要的是，项目中已经确认：

$$c_n(x)=f_n(x+1)$$

于是核心 gcd 从 $\gcd(f,p)$ 统一为 $\gcd(f,c)$ ，也就是：

$$g_n(x)=\gcd(f_n(x), c_n(x))=\gcd(f_n(x), f_n(x+1))$$

这一步的意义很大：算法不再需要旧版本中的 b、p、完整 K 等中间对象，数学主线也变成了非常干净的一对多项式 $f(x)$ 与 $f(x+1)$ 。

5.4 扩展欧几里得给出 q 与 g

在 $GF(2)[x]$ 中对 $f(x)$ 与 $c(x)$ 做扩展欧几里得，得到：

$$s(x)f(x)\oplus q(x)c(x)=g(x), g(x)=\gcd(f(x),c(x))$$

代入 H 后，由于 $f(H)=0$ ，有：

$$q(H)c(H)=g(H)$$

这就是 $O(n^2)$ 算法的核心桥梁。若 $g=1$ ， $q(H)$ 就是 $c(H)$ 的逆；若 $g\neq 1$ ， $q(H)$ 不再是逆，只能把原问题化为 $z=g(H)x$ ，再用 $g(H)$ 的特殊结构反向消元。

6 $O(n^2)$ 主算法：可逆分支与奇异分支

6.1 总体流程

Input : board size n , target vector Y

Output: first-row solution X and full button matrix T

1. Build $f(x)$, $c(x)=f(x+1)$, and the right-hand vector Y .

2. Run extended gcd on f and c :
 $s(x)f(x) \oplus q(x)c(x) = g(x)$
3. Compute $z = q(H)Y$ by repeated left-right diffusion.
4. If $g=1$:
 $X = z$
 Else:
 build or rolling-generate $D = g(H)$
 solve $D X = z$ by reverse elimination.
5. Reconstruct the whole button matrix T from the first row X .

6.2 ApplyPoly: 统一的多项式作用原语

```

ApplyPoly(a, v):
    ans = 0
    cur = v
    for i = 0..deg(a):
        if a[i] = 1:
            ans = ans  $\oplus$  cur
            cur = H * cur
        if cur = 0:
            break
    return ans

```

这个原语至少用于两处： $z=q(H)Y$ ，以及 $d_0=g(H)e_0$ 。若某一步 cur 已经全 0，之后继续乘 H 仍然为 0，因此可以提前停止。这是严格正确的剪枝，不是启发式。

6.3 可逆分支： $g=1$

当 $g(x)=1$ 时，扩展欧几里得给出 $q(x)c(x)=1 \bmod f(x)$ 。代入 H 得 $q(H)c(H)=I$ 。原方程 $c(H)X=Y$ 两边左乘 $q(H)$ ，直接得到：

$$X=q(H)Y$$

因此可逆分支非常干净：只需一次 $\text{ApplyPoly}(q, Y)$ 。共有 n 次扩散，每次处理长度 n 的向量，时间复杂度为 $O(n^2)$ 。这也是 H 矩阵多项式化最甜的地方：矩阵求逆被替换成多项式模逆。

6.4 奇异分支： $g \neq 1$

当 $g(x) \neq 1$ 时， q 不是 c 的逆。此时先计算：

$$z=q(H)Y$$

因为原方程是 $c(H)X=Y$ ，所以：

$$z=q(H)c(H)X=g(H)X$$

令 $D=g(H)$ ，问题变成 $D X=z$ 。 D 不是任意矩阵，而是由 H 的多项式生成，具有可回代结构。设 $r'=\deg(g)$ ， $r=n-r'$ ，则 D 的后 r 行线性无关，并在特定位置形成类似带状上三角的主元结构；前 r' 行可由后 r 行线性叠加得到。于是可以只用后 r 行对 z 做反向消元，得到一个特解。

若要得到全部解，可以在求解前先把 z 与前 r' 行的任意组合相加，再做同样回代。前 r' 行有 $2^{(r')}$ 种组合，因此全部解的数量正是 $2^{(\deg g)}$ 。

6.5 $D=g(H)$ 的带宽与反向消元

$$D[u,v] \neq 0 \Rightarrow |u-v| \leq \deg(g)$$

$$[\max(0, i-r'), \min(n-1, i+r')]$$

这样奇异分支在很多实例中可以从“看起来 $O(n^2)$ 的整行白扫”压到 $O(nr')$ 或接近 $O(nr')$ 的带宽扫描；最坏情况下 $r'=O(n)$ ，仍然是 $O(n^2)$ 。

7 $g(x)$ 的结构、解的数量与秩缺陷

7.1 解的数量

解的数量 $= 2^{deg(g_n)}$, $g_n(x)=gcd(f_n(x),f_n(x+1))$

例如 3×3 时 $deg(g)=0$ ，解唯一； 5×5 时 $deg(g)=2$ ，有 4 个解； 4×4 时 $deg(g)=4$ ，有 16 个解。这里的 $deg(g)$ 同时可以理解为秩缺陷、静默自由度、以及奇异分支中前 r' 行可任意叠加的自由度。

7.2 g_n 与 h_n 的递推结构

$$s_m(x)=f_m(x) \oplus f_{m-1}(x)$$

$$h_m(x)=gcd(s_m(x),s_m(x+1))$$

则有：

$$g_{2m}(x)=h_m(x)^2$$

$$g_{2m+1}(x)=g_m(x)^2, \quad \text{当 } m \bmod 3 \neq 2$$

$$g_{2m+1}(x)=(x^2+x) \cdot g_m(x)^2, \quad \text{当 } m \bmod 3 = 2$$

次数 $d_n=deg(g_n)$ 、 $e_m=deg(h_m)$ 满足：

$$d_{2m}=2e_m$$

$$d_{2m+1}=2d_m, \quad \text{当 } m \bmod 3 \neq 2$$

$$d_{2m+1}=2d_m+2, \quad \text{当 } m \bmod 3 = 2$$

这说明只靠较小的 g_n 本身并不能闭合推出全部较大的 g_n ：奇数分支需要 $m \bmod 3$ ，偶数分支还需要 h_m 家族。更合理的研究方向是维护 g_n 与 h_n 两个家族，形成类似二进制自动机或 2-regular 序列的递推系统。

7.3 $deg(g_n)=n$ 的极端情况

项目中还证明了一个很有辨识度的性质： $d_n=deg(g_n)$ 总是偶数， $0 \leq d_n \leq n$ ，并且 $d_n=n$ 当且仅当 $n=0$ 或 $n=4$ 。证明的核心是把 $deg(g_n)=n$ 等价转写为 $f_n(x)=f_n(x+1)$ 。如果某个首一多项式 $P(x)$ 满足 $P(x+1)=P(x)$ ，则其次数必须为偶数；再结合 doubling 公式，可以推出 $P(n) \Rightarrow P(n/4-1)$ 。

取最小的 $n>4$ 反设，降阶后只可能落到 $n=20$ ；但代 $x=0$ 与 $x=1$ 可直接得到 $f_{20}(0) \neq f_{20}(1)$ ，从而排除 $n=20$ 。于是只剩 $n=0$ 与 $n=4$ 。这个结论解释了为什么 4×4 是一个特别极端的尺寸：第一行自由度全部变成静默自由度。

8 正确性说明与复杂度分析

8.1 正确性骨架

1. 首行压缩正确：从第二行开始，每一行按钮由上一行灯状态唯一决定，因此整盘解与第一行按钮一一对应。

2. 多项式表示正确：首行矩阵可表示为 $c(H)$ ，且 $f(H)=0$ ，所以可在 $GF(2)[x]/(f)$ 中计算后再代入 H 。
3. 扩展欧几里得正确： $s f \oplus q c = g$ ，代入 H 后得到 $q(H)c(H)=g(H)$ 。
4. 可逆分支正确： $g=1$ 时 $q(H)c(H)=I$ ，所以 $X=q(H)Y$ 是原方程唯一解。
5. 奇异分支正确： $g \neq 1$ 时先得 $z=q(H)Y$ ，再由 $z=g(H)X$ 回代求特解；前 $\deg(g)$ 个自由方向给出全部解。

8.2 时间复杂度

完整求解的主要成本来自三个部分：扩展欧几里得 $O(n^2)$ 、 $\text{ApplyPoly}(q, Y)$ $O(n^2)$ 、奇异分支 D 生成与回代最坏 $O(n^2)$ ，最后由首行 X 递推出完整按钮矩阵也需要 $O(n^2)$ 。因此总体为 $O(n^2)$ 。

若只求第一行解，某些子问题仍可能继续优化，例如 half-gcd 可把扩展欧几里得压到 $O(M(n)\log n)$ ，FFT 型多项式乘法有机会进一步逼近 $O(n \log^2 n)$ 。但若要求输出整盘按钮矩阵，输出本身需要 n^2 位，因此总时间复杂度不可能低于 $\Omega(n^2)$ 。这条下界很硬，绕不过去。

8.3 空间复杂度

项目文件已经明确指出，完整矩阵并非都必须保存。 p 可由 c 代替； b 只用于求 p ，可删； k 可由 $g(H)e_0$ 代替； y 、 f 、 c 、 t 、 d 都可以滚动计算。主链只需要保留 y 、 f/c 、 q 、 g 、 z 、 x 等长度为 n 的向量。

因此，如果最终按钮矩阵 t 允许流式输出，算法工作空间可做到 $O(n)$ 。若调用者要求把完整 t 常驻内存，则总空间仍为 $O(n^2)$ ，这不是算法推导失败，而是输出保存方式决定的成本。

9 工程实现演进与后续优化方向

9.1 H 系列实现演进

版本段	主要变化	意义
H - H6	从矩阵法过渡到 f 、 q 、gcd、rev、 z/x 主流程	算法主线开始从高斯消元转向多项式法
H8 - H13	合并 gcd/rev，加入退化处理和 TimeMark	开始细分性能瓶颈，并稳定 gcd + 反向消元框架
H14 - H18	删除 p 、 b 、 k ，滚动 $1/f/c/t/d$ ，只计算 g/d 的必要部分	空间复杂度从完整矩阵逐步压向 $O(n)$ 工作空间
H20 - H22	利用 f 计算 c ，去除冗余变量、TMat 与多余函数	数学主线进一步简化为 $f/c/g/q/z/x$
H23 - H28	尝试 SOS DP、gcd 高位优化、统一 ApplyPoly、TVec 优化，最后删除 BuildC	围绕 q 、 z 、 x 三个热点继续削常数与结构开销

9.2 后续优化方向

6. 扩展欧几里得：朴素 gcd/extgcd 是平方级瓶颈，可研究 half-gcd、分治 extgcd 与快速多项式乘法。
7. 变量替换 $u=x^2+x$ ：由于 $g_n(x)$ 是 x^2+x 的多项式，可尝试把 f 写成 $A(u)+xB(u)$ ，将 gcd 规模约减半。
8. ApplyPoly 加速：统一 $q(H)Y$ 与 $g(H)e_0$ ，可加入全零提前停止，也可进一步研究卷积型快速作用。
9. 奇异分支带宽扫描： $D=g(H)$ 不是满矩阵，回代应只扫有效区间；当 $\deg(g)$ 较小时收益巨大。

10. 商环统一解法：把 $c=gc_1$ 、 $f=gh$ 后，直接在模 h 的商环中解 $c_1u=b_1$ ，有机会整体删除 $D=g(H)$ 与反向叠加。

10 结论

点灯游戏的 $O(n^2)$ 求解并不是单纯的代码优化，而是多层结构压缩的结果：第一层把全局 n^2 变量压到首行 n 变量，第二层把首行矩阵压成 H 的多项式，第三层用 $f(H)=0$ 把矩阵方程压进 $GF(2)[x]/(f)$ ，第四层用 $\gcd(f,c)$ 区分可逆与奇异，再用 $q(H)Y$ 和 $g(H)X$ 完成求解。

从算法角度看，完整输出 $n \times n$ 按钮矩阵时， $O(n^2)$ 已经达到输出下界。之后真正值得研究的是两个方向：一是把内部的 $\gcd$ 、 q 、 ApplyPoly 和奇异回代继续做低常数或亚平方优化；二是如果只关心解数量、秩缺陷或首行解，则进一步追求 $O(n \log n)$ 或 $O(n \log^2 n)$ 的多项式算法。

从数学角度看， $g_n(x)=\gcd(f_n(x),f_n(x+1))$ 是整套算法的灵魂。它既决定解空间维数，也决定奇异分支的自由度，还暴露了 x^2+x 、 g/h 双家族递推等更深结构。换句话说，点灯游戏表面上是点灯，骨子里是 $GF(2)$ 多项式在下棋。

参考文献与项目资料

- [1] 项目文件：README.md，点灯游戏（Lights Out）求解器·算法说明。
- [2] 项目文件：点灯游戏 manim.txt / @原稿 manim.txt， $O(n^2)$ 解法视频脚本与算法总结。
- [3] 项目文件：@空间复杂度优化.txt，滚动向量与 $O(n)$ 工作空间说明。
- [4] 项目文件：@时间复杂度优化.txt， q 、 z 、 x 热点与 half-gcd / 带宽回代优化路线。
- [5] 项目文件：@关于 $g(x)$ 有以下公式：.txt， g_n 、 h_n 与次数递推公式。
- [6] 项目文件：@证明 $\deg(g(n))=n$.txt， $\deg(g_n)=n$ 当且仅当 $n=0$ 或 4 的证明。
- [7] Jaap Scherphuis, The Mathematics of Lights Out. <https://www.jaapsch.net/puzzles/lomath.htm>
- [8] Andries E. Brouwer, Button Madness / Lights Out on rectangular boards.
<https://aeb.win.tue.nl/ca/madness/madrect.html>
- [9] Henrik Eriksson, Kimmo Eriksson, Jonas Sjöstrand, Note on the Lights Out game. arXiv:math/0411201.
- [10] Rudolf Fleischer, Jiajin Yu, A Survey of the Game Lights Out. DOI: 10.1007/978-3-642-40273-9_13.
- [11] Martin Kreh, The Game of Lights Out and its Variations. American Mathematical Monthly, 2017.
- [12] OEIS A075462 / A159257 / A076436, $n \times n$ 点灯游戏解数、秩缺陷与唯一解尺寸相关序列。